ENSEEIHT

Développement formel de systèmes
Rapport

Dynamic Source Routing (DSR) Protocol

Élèves :
Ninon Autefage-Pinies
Yige YANG

Enseignant :
Dupont Guillaume

29 avril 2026

Table des matières

1 Introduction **3**

2 Présentation du projet **4**

3 Raffinement **5**

3.1 Modèle initial `DSR_0` . . . 5
3.1.1 Contexte et exemple . . . 5
3.1.2 Variables et invariants . . . 5
3.1.3 Initialisation . . . 5
3.1.4 Évenements . . . 5
3.1.5 Cycle de vie des messages . . . 6
3.2 Premier raffinement `DSR_1` . . . 7
3.2.1 Contexte et exemple . . . 7
3.2.2 Variables et invariants de typage . . . 7
3.2.3 Initialisation . . . 7
3.2.4 Évenements . . . 7
3.3 Deuxième raffinement `DSR_2` . . . 7
3.3.1 Contexte et exemple . . . 8
3.3.2 Variables et invariants de typage . . . 8
3.3.3 Initialisation . . . 8
3.3.4 Évenements . . . 8
3.4 Troisième raffinement `DSR_3` . . . 9
3.4.1 Contexte et exemple . . . 9
3.4.2 Variables et invariants . . . 9
3.4.3 Initialisation . . . 9
3.4.4 Évenements . . . 9
3.5 Quatrième raffinement `DSR_4` . . . 11
3.5.1 Introduction des numéros de séquence . . . 11
3.5.2 Nouvelle variable et invariant . . . 12
3.5.3 Initialisation . . . 12
3.5.4 Lancement d'une requête `RREQ` . . . 12
3.5.5 Nettoyage des informations de découverte . . . 13
3.5.6 Rôle de `DSR_4` dans le développement . . . 13
3.6 Cinquième raffinement `DSR_5` . . . 13
3.6.1 Introduction de la maintenance de routes . . . 13
3.6.2 Nouvelles variables et invariants . . . 13
3.6.3 Initialisation . . . 13
3.6.4 Détection d'une erreur de lien . . . 14
3.6.5 Traitement de l'erreur à la source . . . 14
3.6.6 Exemple de scénario . . . 14
3.6.7 Rôle de `DSR_5` dans le développement . . . 15

4 Améliorations et évolutions **16**

4.1 Améliorations du modèle actuel . . . 16
4.2 Evolutions possibles . . . 16

5 Bilan **17**
5.1 Difficultés rencontrées 17
5.2 Bilans personnels 17

6 Conclusion **18**

Table des figures

1 Envoi simple 6
2 Perte de mesage 6
3 Envoi complet 7
4 Exemple de table de routage (`Cache`) 8
5 Exemple d'une diffusion de `RREQ` 10
6 Exemple d'une réponse `RREP` 11
7 Scénario de perte d'une `RREQ` avec un site "pont" 11

1 Introduction

Les réseaux ad hoc sans fil apparaissent aujourd'hui dans de nombreux contextes réels : réseaux de capteurs, communications entre robots mobiles, situations d'urgence ou d'intervention où aucune infrastructure fixe n'est disponible. Dans ce type de réseau, les nœuds doivent coopérer pour relayer les paquets, la topologie peut changer à tout moment et il n'est pas réaliste de s'appuyer sur des tables de routage globales mises à jour périodiquement. Le protocole *Dynamic Source Routing* (DSR) répond à cette problématique en permettant à chaque source de choisir explicitement la route suivie par ses paquets, et en ne déclenchant la découverte et la maintenance des routes que lorsque cela est nécessaire.

Dans ce projet, nous avons cherché à traduire ces idées dans un cadre formel en utilisant Event-B. L'objectif n'est pas seulement de reproduire le comportement du protocole, mais aussi de mettre en évidence, par les invariants et les preuves, les propriétés essentielles que l'on attend d'un tel système : absence de blocage, cohérence de la topologie vue localement par les nœuds et capacité à transmettre correctement les messages lorsque l'environnement devient stable.

La modélisation nous a également conduits à soigner particulièrement les conditions de garde des événements. Celles-ci doivent être suffisamment fortes pour empêcher les comportements incohérents (par exemple l'envoi d'un message sur un lien inexistant), tout en laissant possible l'évolution normale du système. Pour tester ces choix, nous avons exploré différents scénarios d'utilisation, comme l'envoi de messages dans un petit réseau de trois nœuds représentant un mini-réseau de capteurs, ou la façon dont une panne de lien doit être signalée à la source et traitée pour éviter d'utiliser une route devenue invalide.

Pour structurer cette réflexion, nous avons adopté une démarche de raffinements successifs. Le modèle initial `DSR_0` décrit un protocole de communication minimal où l'on peut créer et supprimer des connexions, produire, envoyer, recevoir ou perdre des messages sans notion explicite de route. Il fournit un socle abstrait sur lequel il est possible de raisonner sur la circulation des messages dans un réseau dynamique.

Le premier raffinement `DSR_1` introduit une vision plus proche d'un réseau réel en distinguant, pour chaque message, un site source et un site destination, ainsi que deux états possibles : message stocké sur un nœud ou message en transit sur un lien. Ce niveau correspond à l'architecture *store-and-forward*, où chaque nœud joue à la fois le rôle de point de stockage et de relais.

Le deuxième raffinement `DSR_2` introduit une table de routage locale `Cache` et une variable `Route` portée par les messages prêts à être transmis. Le chemin choisi par la source est ainsi représenté explicitement à partir des informations disponibles dans sa vue locale du réseau, ce qui rapproche déjà le modèle du principe de routage par source.

Les raffinements suivants prolongent cette idée en intégrant progressivement les mécanismes de découverte et de maintenance décrits dans le sujet. La machine `DSR_3` introduit un mécanisme explicite de *Route Discovery* basé sur des requêtes de route `RREQ` diffusées dans le réseau et sur la construction incrémentale des chemins. La machine `DSR_4` enrichit ce mécanisme avec des numéros de séquence et des structures auxiliaires permettant d'ignorer les requêtes déjà vues et de limiter les redondances.

Enfin, la machine `DSR_5` intègre un mécanisme de *Route Maintenance* qui permet de détecter les ruptures de liens, de générer des paquets d'erreur de route `RERR` vers la source et de déclencher l'adaptation ou la redécouverte de routes. L'ensemble des modèles `DSR_0` à `DSR_5` permet ainsi de passer progressivement d'une abstraction très simple à une représentation plus fidèle d'un protocole de routage pour réseaux ad hoc, tout en conservant un contrôle formel sur les propriétés de sûreté et de fonctionnement.

2 Présentation du projet

Le projet s'inscrit dans le cadre du module *Développement formel de systèmes* et vise à modéliser le protocole de routage *Dynamic Source Routing* (DSR) pour des réseaux ad hoc sans fil, à l'aide de la méthode des raffinements Event-B et de la plateforme Rodin. Le protocole DSR, tel qu'il est présenté dans le sujet de projet, repose sur deux mécanismes principaux, *Route Discovery* et *Route Maintenance*, qui permettent à un nœud source de découvrir dynamiquement des routes vers une destination et de maintenir ces routes en présence de changements de topologie.

L'idée générale de la modélisation est de partir d'un modèle très abstrait de communication de messages, puis d'introduire progressivement les éléments caractéristiques de DSR. Plus précisément, le développement tient compte des aspects suivants :

— l'architecture *store-and-forward*, où les messages sont soit stockés sur un site, soit en transit sur un lien ;
— une table de routage locale, utilisée pour déterminer la prochaine étape d'un message vers sa destination ;
— un mécanisme de découverte de routes par requêtes de diffusion lorsque la table de routage ne contient pas encore de chemin ;
— la gestion explicite des erreurs de route lorsque des liens deviennent indisponibles.

Cette progression est structurée en une suite de machines Event-B `DSR_0`, `DSR_1`, `DSR_2`, `DSR_3`, `DSR_4` et `DSR_5`. Chaque machine raffine la précédente en ajoutant de nouvelles fonctionnalités du protocole, et la validation est réalisée à chaque étape par :

— des invariants de sûreté exprimant les propriétés essentielles du système ;
— des obligations de preuve prouvées dans Rodin ;
— des animations et vérifications de scénarios typiques à l'aide de ProB.

Les contextes `ctx_0` et `ctx_1` introduisent les ensembles abstraits de sites (`SITES`) et de messages (`MESSAGES`) ainsi que les hypothèses sur l'environnement réseau (nombre fini de nœuds, liens dirigés pouvant apparaître ou disparaître, etc.). Les contextes `exemple_0` et `exemple_1` fournissent un petit scénario instancié avec trois nœuds nommés et un nombre fini de messages, afin de permettre une exploration systématique des comportements avec ProB et de vérifier concrètement les propriétés modélisées.

Le rapport présentera l'architecture complète de ces modèles, en se concentrant sur les machines actuellement réalisées (`DSR_0` à `DSR_2`) et en décrivant également la conception et le rôle prévus pour `DSR_3`, `DSR_4` et `DSR_5` dans le développement global.

3 Raffinement

3.1 Modèle initial DSR_0

Le modèle DSR_0 constitue le point de départ du développement. Il correspond au modèle initial du sujet, dont le rôle est de capturer le comportement de base d'un réseau ad hoc : apparition et disparition de liens, circulation de messages entre nœuds, et perte de messages lorsque la topologie change, sans encore introduire de notion de source, de destination ou de routage.

3.1.1 Contexte et exemple

La machine DSR_0 voit le contexte abstrait ctx_1, qui introduit l'ensemble SITES des nœuds du réseau, supposé fini. Le contexte exemple_1 spécialise ensuite ce cadre en instanciant SITES par trois constantes A, B et C, reliées par l'axiome

```
partition(SITES,{A},{B},{C}),
```

ce qui garantit que le réseau étudié dans les animations est composé exactement de trois sites distincts. Ce petit exemple concret facilite l'exploration systématique des comportements avec ProB.

3.1.2 Variables et invariants

La machine DSR_0 introduit cinq variables :

— Reseau $\subseteq$ SITES $\times$ SITES représente l'ensemble des liens dirigés qui existent actuellement entre les sites. Si 2 sites sont connectés, ils le sont dans les deux sens.
— Messages $\subseteq$ MESSAGES est l'ensemble des messages existants dans le système.
— SurSite $\in$ Messages $\rightarrow$ SITES associe à chaque message stocké, le site où il se trouve.
— SurReseau $\in$ Messages $\rightarrow$ (SITES $\times$ SITES) associe à chaque message en transit le lien orienté (s_1, s_2) sur lequel il circule.
— Perdus $\subseteq$ Messages regroupe les messages qui ont été perdus. Un message est perdu s'il est en transit sur un lien qui a été supprimé.

L'invariant clé

```
partition(Messages, dom(SurSite), dom(SurReseau), Perdus)
```

impose que chaque message est dans un seul des trois états possibles : stocké sur un site, en transit sur un lien ou perdu.

3.1.3 Initialisation

— Reseau est initialisé à tous les couples de sites possibles sauf les boucles (s, s), ce qui modélise un réseau complet simple.
— Messages, SurSite, SurReseau et Perdus sont initialisés à l'ensemble vide.

3.1.4 Évenements

Deux événements permettent de modifier la topologie :

— CreerConnexion : ajoute symétriquement les liens (s_1, s_2) et (s_2, s_1) à Reseau, sous la condition que ces liens n'existent pas encore et que $s_1 \neq s_2$.
— PerdreConnexion: retire ces deux liens de Reseau.

3.1.5 Cycle de vie des messages

L'échange simple (Fig. 1) et la perte de messages (Fig. 2) est décrit par les cinq événements suivants :

- `ProduireMessage:` crée un nouveau message `mss` (choisi dans `MESSAGES` et absent de `Messages`) et le place sur un site `src`. Le message est alors dans l'état « stocké sur un site ».
- `Envoyer:` modélise l'envoi d'un message `mss` d'un site `src` vers un site `dst` : le message doit se trouver sur `src` et le lien (`src`, `dst`) doit exister dans le `Reseau`. L'événement retire `mss` de `SurSite` et l'ajoute à `SurReseau` sur le lien (`src`, `dst`).
- `Recevoir:` modélise la réception de `mss` par `dst` depuis le lien (`src`, `dst`) : le message est retiré de `SurReseau` et ajouté à `SurSite` sur le site `dst`.
- `PerdreMessage:` gère la perte d'un message en transit. Si `mss` est dans le domaine de `SurReseau` et que `SurReseau(mss)` n'appartient plus à `Reseau`, l'événement supprime `mss` de `SurReseau` et l'ajoute à `Perdus`, conformément à l'hypothèse de perte de paquets sur un lien tombé en panne.
- `RecevoirTimeout` permet de modéliser un mécanisme abstrait de timeout côté émetteur : lorsqu'un message appartient à `Perdus`, on peut le retirer de cet ensemble et le replacer sur un site `src` dans `SurSite`, préparant ainsi un éventuel nouvel envoi.

À ce niveau de raffinement, le site émetteur `src` est choisi de manière abstraite, la notion de source réelle du message n'étant pas encore introduite. Ce premier modèle se concentre donc sur le support de communication et les aléas du réseau, sans aucune information de routage. Il fournit une base simple mais déjà rigoureusement spécifiée, sur laquelle les raffinements suivants (`DSR_1` à `DSR_5`) viendront ajouter la structure *store-and-forward*, la table de routage locale, puis les mécanismes de découverte et de maintenance des routes.

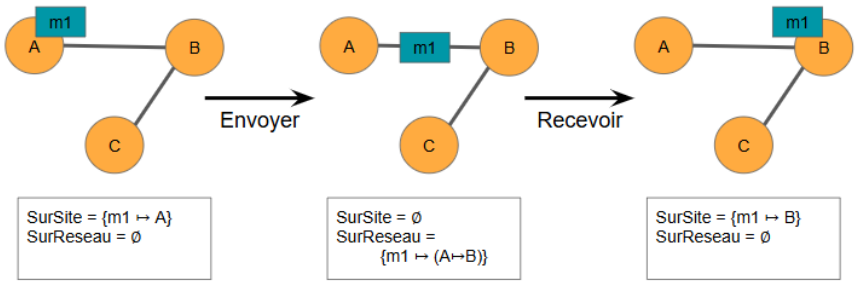

FIGURE 1 – Envoi simple

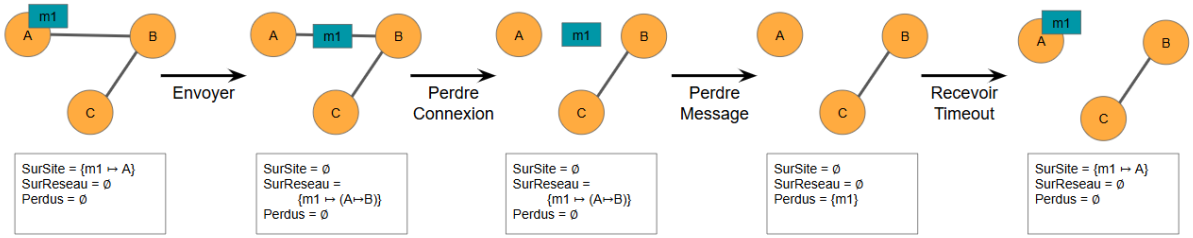

FIGURE 2 – Perte de mesage

3.2 Premier raffinement DSR_1

Le modèle DSR_1 est le premier raffinement et permet de rendre l'échange de messages plus concret en différenciant le site emetteur, le site recepteur et les sites intermédiaires pour les messages (Fig 3).

3.2.1 Contexte et exemple

La machine DSR_1 raffine la machine DSR_0. Le contexte ne change pas.

3.2.2 Variables et invariants de typage

Deux nouvelles variables ont été ajoutées :
— Source $\in$ Messages $\rightarrow$ SITES qui associe à tout message, son unique site source.
— Destination $\in$ Messages $\rightarrow$ SITES qui associe à tout message, son unique site destination.

3.2.3 Initialisation

Source et Destination sont initialisés à l'ensemble vide (puisqu'il n'y a aucun message dans le réseau initialement).

3.2.4 Évenements

— EnvoyerDepart et Envoyer raffinent l'évenement abstrait Envoyer. EnvoyerDepart représente l'envoi d'un message depuis sa source, tandis qu'Envoyer représente l'envoi d'un message depuis un noeud quelconque du réseau.
— RecevoirFinal et Recevoir raffinent l'évenement abstrait Recevoir. RecevoirFinal représente la reception d'un message sur sa destination, tandis qu'Recevoir représente la réception d'un message sur un noeud quelconque du réseau.
— RecevoirTimeout contraint le noeud recevant le timeout à être la source du message perdu.

Ce premier raffinement permet d'introduire les notions de source et de destination des messages. Il s'agit aussi de distinguer l'envoi (resp. la réception) de messages depuis (resp. vers) le site source (resp. destination).

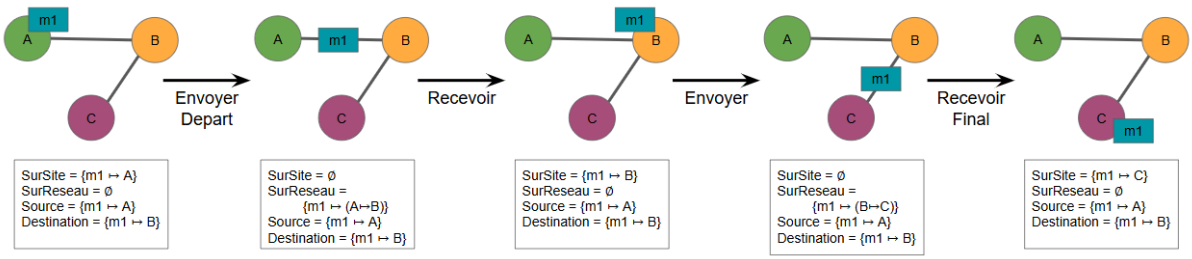

FIGURE 3 – Envoi complet

3.3 Deuxième raffinement DSR_2

Le modèle DSR_2 constitue le deuxième raffinement et introduit la notion de route et l'ajout de table de routage locale (Fig 4).

A	B	{(A↦B)}
	C	{(A↦B), (B↦C)}
B	A	{(B↦A)}
	C	{(B↦C)}
C	A	{(C↦B)(B↦A)}
	B	{(C↦B)}

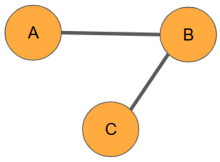

FIGURE 4 – Exemple de table de routage (`Cache`)

3.3.1 Contexte et exemple

La machine `DSR_2` raffine la machine `DSR_1`. Le contexte ne change pas.

3.3.2 Variables et invariants de typage

Quatre nouvelles variables ont été ajoutées :

- $\texttt{Cache} \in \texttt{SITES} \rightarrow (\texttt{SITES} \nrightarrow (\texttt{SITES} \rightarrowtail\!\!\!\!\rightarrow \texttt{SITES}))$ représente le cache local d'un site contenant les routes vers les différents sites.
- $\texttt{Route} \in \texttt{Prets} \rightarrow (\texttt{SITES} \nrightarrow \texttt{SITES}))$ associe chaque message à une route reliant sa source à sa destination.
- $\texttt{Attente} \subseteq \texttt{Messages}$ est l'ensemble des messages ne possédant pas encore de route.
- $\texttt{Prets} \subseteq \texttt{Messages}$ est l'ensemble des messages possédant une route.

L'invariant

$$\texttt{partition(Messages, Prets, Perdus, Attente)}$$

impose que chaque message peut être en attente, pret ou perdu.

3.3.3 Initialisation

`Attente`, `Prets` et `Route` sont initialisés à l'ensemble vide (puisqu'il n'y a aucun message dans le réseau initialement). `Cache` est initialisé de manière à ce que, à chaque site soient associées les routes vers tous les autres sites.

$$\texttt{Soient les sites \{A, B, C\}, Cache} = \left\{\begin{array}{l} A \mapsto \{B \mapsto \{(A \mapsto B)\},\ C \mapsto \{(A \mapsto C)\}\}, \\ B \mapsto \{A \mapsto \{(B \mapsto A)\},\ C \mapsto \{(B \mapsto C)\}\}, \\ C \mapsto \{A \mapsto \{(C \mapsto A)\},\ B \mapsto \{(C \mapsto B)\}\} \end{array}\right\}$$

Nous avons choisi cette implémentation car nous sommes parties du principe que les sites connaissent leur voisins, or à l'initialisation, tous les sites sont voisins.

3.3.4 Évenements

- `PerdreConnexion` est raffiné pour supprimer les liens directs dans les caches des sites concernés par la perte de connexion.
- `AjouterRouteCache` est un nouvel evénement abstrait qui permet d'ajouter une route au cache si la destination de la route n'appartient pas au cache de la source.
- `RetirerRouteCache` représente le fait qu'un site retire du cache une route simple vers un de ses voisins si la connection entre lui et son voisin n'existe plus dans le réseau.
- Dans `ProduireMessage`, le nouveau message est placé en attente.
- `ChoisirRoute` permet d'associer une route à un message (qui passe alors dans `Prets`) lorsque la route vers la destination est dans le cache de la source du message.

— `Envoyer`, `EnvoyerDepart`, `Recevoir` et `RecevoirFinal` ont une nouvelle garde qui s'assure que le message est dans `Prets` et que les sites sont dans la route du message.
— `PerdreMessage` et `RecevoirTiemout` s'assurent de préserver l'invariant liant `Prets`, `Attente` et `Perdus`.

Ici nous avons choisi de modéliser le fait que chaque site connait ses voisins et met à jour son cache en fonction de l'évolution du réseau, on suppose que si un site perd la connexion avec son voisins alors il existe un mécanisme lui permettant de savoir que la connexion a été perdue.

Ce second raffinement ajoute les notions de cache local et de route au modèle précédent. Néanmoins, à ce stade il n'y a pas de mécanisme pour chercher une route, les routes qui sont ajoutées dans le cache sont purement théorique.

3.4 Troisième raffinement `DSR_3`

Le modèle `DSR_3` est le troisième raffinement et met en place le système de *Route Discovery* avec les requêtes de routes (Fig.5) et les réponses (Fig.6).

3.4.1 Contexte et exemple

La machine `DSR_3` raffine la machine `DSR_2`. Le contexte ne change pas.

3.4.2 Variables et invariants

Sept nouvelles variables ont été ajoutées :
— `RREQ` $\subseteq$ `Attente` est un sous ensemble des messages en attente, représentant les requêtes de route.
— `RoutesRREQ` $\in$ `RREQ` $\rightarrow \mathcal{P}(\mathcal{P}(\text{SITES} \times \text{SITES}))$ associe chaque requête à un ensemble de routes.
— `SeenRREQ` $\in$ `SITES` $\rightarrow \mathcal{P}(\text{Messages})$ associe chaque site à l'ensemble des requêtes de route qu'il a reçues.
— `SurReseauRREQ` $\in$ `RREQ` $\rightarrow \mathcal{P}(\text{SITES} \times \text{SITES})$ associe à chaque requête tous les liens sur lesquels se trouve la requête (les requêtes étant diffusées, elles peuvent être sur plusieurs liens à la fois).
— `RREP` $\subseteq$ `Attente` est un sous ensemble des messages en attente, représentant les réponses aux requêtes de route.
— `RouteRREP` $\in$ `RREP` $\rightarrow \mathcal{P}(\text{SITES} \times \text{SITES})$ associe chaque réponse à une route.
— `SurReseauRREP` $\in$ `RREP` $\nrightarrow (\text{SITES} \times \text{SITES})$ associe chaque réponse au lien sur lequel elle se trouve (une requête ne peut se trouver que sur un seul lien à la fois).

3.4.3 Initialisation

Les nouvelles variables sont initialisées à l'ensemble vide, à l'exception de `SeenRREQ` qui est initialisé de sorte à ce que, à chaque site soit associé un ensemble vide (les sites n'ont reçu aucune requête).

3.4.4 Évenements

— `LancerRREQ` est un événement qui crée une nouvelle requête de route pour un message qui n'a pas encore de route associée et dont il n'existe pas de route dans le cache local de la source. Le site source envoie la requête à tous ses voisins.
— `DiffuserRREQ` est l'événement qui permet à un site intermédiaire qui n'a pas déjà traité la RREQ, de la recevoir et de la diffuser aussitôt à ses voisins.

— `RejeterRREQ` simule la réception d'une RREQ par un site qui l'a déjà traité, la RREQ est retirée du réseau (1 exemplaire uniquement) et rien n'est diffusé.
— `CreerRREP` représente la réception d'une RREQ par le site destination d'un message. Simultanément à la réception, le site crée une RREP auquel la route reçue est associée.
— `EnvoyerRREQ` permet d'envoyer une RREP nouvellement créée depuis le site destination.
— `TransmettreRREQ` permet à un site intermédiaire de recevoir et d'envoyer au site suivant (dans la route de RREP) une RREP.
— `RecevoirRREQ` est réalisé lorsque le site source du noeud recoit la RREP d'un message.
— `PerdreRREQ` représente la perte d'une requête de route lorsqu'une RREQ est sur une connexion qui disparait.
— `PerdreRREP` représente la perte d'une réponse de route lorsqu'une RREP est sur une connexion qui disparait.
— `RecevoirTiemoutRREQ` est réalisé lorsque le site source du noeud recoit la RREP d'un message.

Remarques :

Perdre une `RREQ` n'implique pas que le site source va recevoir un timeout, il faut que *tous* les exemplaires de la `RREQ` soient perdus ou rejetés pour recevoir un timeout. Cela signifie que si un site "pont" recoit une `RREQ` qui est ensuite perdue alors le site source finira par recevoir un tiemout même si il y a d'autres `RREQ` en circulation car nous avons choisi de ne pouvoir traiter qu'une seule `RREQ` par site et par message (Fig. 7).

Dans ce modèle nous avons choisi de faire en une seule étape les équivalents des événements `Envoyer` et `Recevoir` pour les `RREQ` et `RREP`. En effet,il n'y a pas de notion de stockage sur les sites, les requêtes et réponses de route sont uniquement représentées sur le réseau.

Pour simplifier le modèle et réduire la taille des ensembles et des états possibles nous avons choisi de "nettoyer" les ensembles concernant les `RREQ` et les `RREP` une fois la réponse arrivée au site source. Sans cela, les `RREQ` seraient toutes rejetées jusqu'à ce que tous les sites aient reçu tous les exemplaires de la `RREQ`.

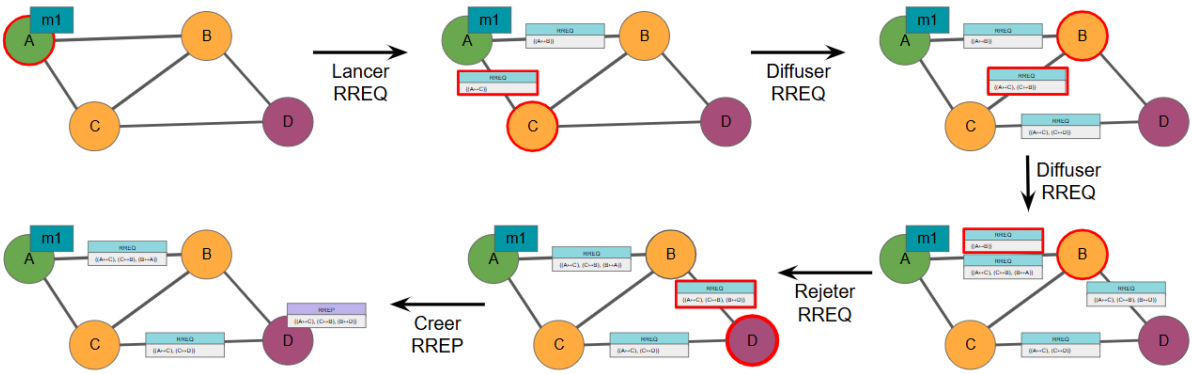

FIGURE 5 – Exemple d'une diffusion de `RREQ`

Les éléments en rouge à l'étape `n` font partie des `EventParameter` de l'`Event` permettant de passer de l'étape `n` à l'étape `n+1`.

Ce troisième raffinement modélise le système de découverte de route, mais il est possible qu'un message reste bloqué si une connexion est perdue lorsque le message a une route initialisée et qu'il est stocké sur un site (il restera bloqué jusqu'à ce que la connexion perdue se recrée).

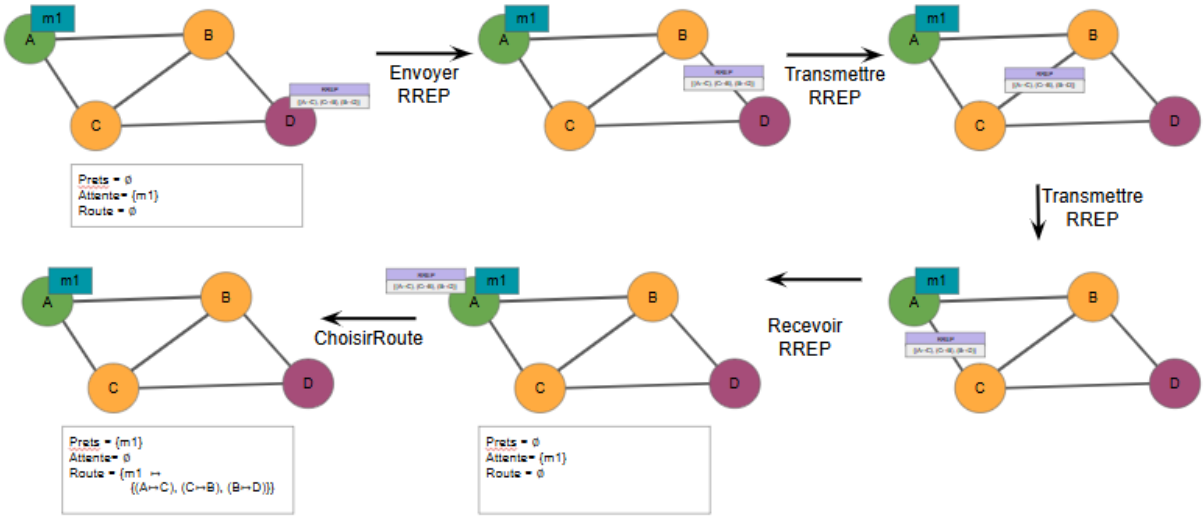

FIGURE 6 – Exemple d'une réponse RREP

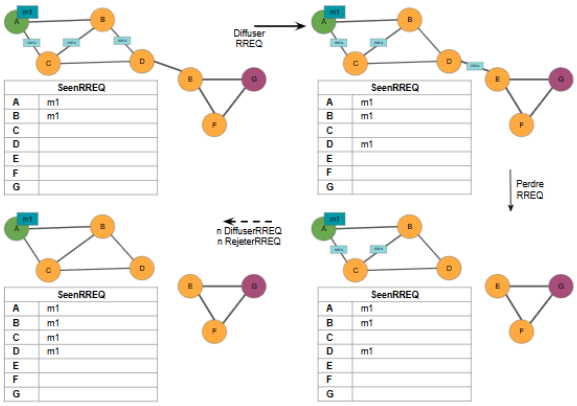

FIGURE 7 – Scénario de perte d'une RREQ avec un site "pont"

3.5 Quatrième raffinement DSR_4

3.5.1 Introduction des numéros de séquence

La machine DSR_4 raffine DSR_3 en introduisant des numéros de séquence associés aux requêtes de route RREQ. Comme pour DSR_5, il s'agit d'une amélioration du modèle : la structure de la table de routage Cache n'est pas modifiée, mais une information supplémentaire est ajoutée pour suivre plus finement les requêtes, sans changer le comportement fonctionnel de base. [1]

Cette extension permet d'identifier plus précisément les requêtes émises par les nœuds, de filtrer les requêtes redondantes pour un même couple source–destination, et prépare le modèle à des variantes où plusieurs routes peuvent être envisagées pour une même destination, sans modifier la structure des raffinements précédents.

Un petit scénario illustre l'intérêt des numéros de séquence :

— Le nœud A souhaite une route vers B et envoie une première requête RREQ avec le numéro de séquence 1.

— Plus tard, la route échoue et A renvoie une nouvelle requête vers B, cette fois avec le

1. « Cinquième raffinement. Introduction de numéros de séquence pour le suivi des informations des paquets de demande de nouveaux itinéraires. »

numéro de séquence 2.
— Lorsqu'un nœud intermédiaire reçoit une requête pour `A` → `B`, il consulte le numéro : si le numéro est 2, il sait qu'il s'agit d'une requête plus récente que celle portant le numéro 1, et il peut conserver l'information associée à 2 et ignorer l'ancienne associée à 1.

Ce comportement peut être résumé dans le tableau suivant :

Requête	Source	Destination	Numéro de séquence	Interprétation
`RREQ_1`	A	B	1	Première demande de route
`RREQ_2`	A	B	2	Demande plus récente (après échec de la route précédente)

Ainsi, même si `Cache` ne stocke qu'une seule route par destination dans notre développement, l'introduction de `RREQ_seq` offre une solution possible pour distinguer les demandes successives d'une même source.

3.5.2 Nouvelle variable et invariant

La machine `DSR_4` reprend l'ensemble des variables de `DSR_3` et introduit la fonction partielle suivante :

— $RREQ_seq \in RREQ \rightarrow \mathbb{N}$, qui associe à chaque requête de route un numéro de séquence.

L'invariant suivant est ajouté afin de typer cette fonction :

$$invR1 : RREQ_seq \in RREQ \rightarrow \mathbb{N}$$

Il garantit que seules les requêtes appartenant à l'ensemble `RREQ` possèdent un numéro de séquence et que ce numéro est un entier naturel.

3.5.3 Initialisation

L'événement `INITIALISATION` de `DSR_4` raffine celui de `DSR_3` et conserve les valeurs initiales des variables existantes. Une action supplémentaire est ajoutée pour initialiser la nouvelle fonction :

$$RREQ_seq := \emptyset$$

Ainsi, aucune requête ne possède de numéro de séquence à l'état initial.

3.5.4 Lancement d'une requête `RREQ`

L'événement `LancerRREQ` est raffiné afin d'attribuer un numéro de séquence à chaque nouvelle requête. Un paramètre supplémentaire $n \in \mathbb{N}$ est introduit, avec une garde imposant que la requête ne soit pas déjà dans le domaine de `RREQ_seq`.

Dans la partie `THEN`, après les mises à jour existantes, l'action suivante est ajoutée :

$$RREQ_seq := RREQ_seq \cup \{req \mapsto n\}$$

3.5.5 Nettoyage des informations de découverte

Lorsque la découverte de route aboutit et que la requête quitte l'ensemble `RREQ`, les informations associées doivent être supprimées de manière cohérente. L'événement `ChoisirRoute` est raffiné afin de retirer la requête concernée des ensembles et fonctions associés.

Une action supplémentaire est ajoutée pour supprimer le numéro de séquence :

$$RREQ_seq := \{mss\} \lhd RREQ_seq$$

Cette action garantit la préservation de l'invariant `invR1`.

3.5.6 Rôle de `DSR_4` dans le développement

Le raffinement `DSR_4` enrichit le modèle par l'ajout d'une information de séquencement des requêtes, sans modifier le comportement fonctionnel de la découverte de route introduite dans `DSR_3`. Cette extension permet de rapprocher le modèle des descriptions du protocole tout en limitant l'impact sur la complexité et la charge de preuve.

3.6 Cinquième raffinement `DSR_5`

3.6.1 Introduction de la maintenance de routes

La machine `DSR_5` raffine `DSR_4` en ajoutant un mécanisme abstrait de maintenance de routes. L'objectif est de modéliser la détection des ruptures de liens lors de l'acheminement des paquets et la réaction de la source face à ces erreurs, tout en conservant le comportement fonctionnel de découverte et de transmission de route déjà introduit dans les modèles précédents.

3.6.2 Nouvelles variables et invariants

En plus des variables héritées de `DSR_4`, la machine `DSR_5` introduit deux nouvelles variables :

- `RERR` : ensemble des messages ayant fait l'objet d'une erreur de route.
- `ErrLink` : fonction partielle associant à certains messages de `RERR` le lien défaillant (src, dst) sur lequel l'erreur a été détectée.

Ces variables sont typées par les invariants suivants :

$$inv1 : RERR \subseteq Messages$$
$$inv2 : ErrLink \in RERR \rightharpoonup (SITES \times SITES)$$

Le premier invariant garantit que seuls des messages existants peuvent être marqués en erreur. Le second impose qu'un message ne possède au plus qu'un lien erroné associé et que ce lien relie deux sites du réseau.

3.6.3 Initialisation

L'événement `INITIALISATION` de `DSR_5` raffine celui de `DSR_4` et conserve les valeurs initiales des variables précédentes. Deux actions supplémentaires sont ajoutées pour initialiser les nouvelles structures :

$$RERR := \emptyset \qquad ErrLink := \emptyset$$

Ainsi, aucun message n'est initialement en erreur et aucun lien défaillant n'est mémorisé.

3.6.4 Détection d'une erreur de lien

L'événement `DetecterErreurLien` raffine l'événement abstrait `PerdreMessage`. Il représente le cas où un message en transit utilise un lien qui n'appartient plus au réseau (rupture de connexion).

Les gardes principales imposent que :

— le message `mss` se trouve dans le domaine de `SurReseau` et transite sur un lien $src \mapsto dst$;
— ce lien n'appartient plus à `Reseau` ;
— le message est encore dans `Prets` et n'appartient pas à `Perdus` ni au domaine de `ErrLink`.

La partie `THEN` reprend le comportement de `PerdreMessage` :

— suppression du message de `SurReseau` ;
— ajout du message dans `Perdus` ;
— retrait de la route et du statut prêt (`Route`, `Prets`).

Deux actions supplémentaires réalisent la mémorisation de l'erreur :

$$RERR := RERR \cup \{mss\}$$
$$ErrLink := ErrLink \cup \{mss \mapsto (src \mapsto dst)\}$$

Ainsi, chaque message perdu à cause d'un lien rompu est marqué dans `RERR` et le lien fautif associé est mémorisé.

3.6.5 Traitement de l'erreur à la source

La machine `DSR_5` introduit également un nouvel événement `TraiterRERR_A_Source`, qui ne raffine aucun événement précédent. Il modélise de manière abstraite la réaction du nœud source lorsqu'un message a été marqué en erreur.

Les gardes imposent notamment que :

— `mss` appartient à `RERR` ;
— le message n'est ni dans `Attente`, ni présent sur un site ou sur le réseau (`SurSite`, `SurReseau`) ;
— `src` est la source du message `Source(mss)`.

Les actions suivantes remettent le message en circulation à partir de la source :

— $Perdus := Perdus \setminus \{mss\}$;
— $SurSite := SurSite \cup \{mss \mapsto src\}$;
— $Attente := Attente \cup \{mss\}$;
— $RERR := RERR \setminus \{mss\}$.

Cet événement fournit une abstraction du mécanisme de maintenance de routes : la source peut réessayer d'acheminer le message en relançant, si nécessaire, la découverte de route.

3.6.6 Exemple de scénario

Le comportement de `RERR` et `ErrLink` peut être illustré par le scénario suivant :

Étape	Description
1	Le nœud `A` envoie un message `m1` vers `C` en utilisant la route `A` $\rightarrow$ `B` $\rightarrow$ `C`. Le message est en transit sur le lien `A` $\rightarrow$ `B`, c'est-à-dire $m1 \in \mathrm{dom}(SurReseau)$ et $SurReseau(m1) = A \mapsto B$.
2	La liaison `A` $\rightarrow$ `B` tombe en panne et n'appartient plus à `Reseau`. L'événement `DetecterErreurLien` est activé ; après son exécution, on a $m1 \in Perdus$, $m1 \in RERR$ et $ErrLink(m1) = (A \mapsto B)$. Le message n'est plus dans `SurReseau` ni dans `Prets`.
3	Dans un second temps, la source `A` traite cette erreur via l'événement `TraiterRERR_A_Source`. Après cet événement, le message est replacé au niveau de la source et remis en attente : $m1 \in Attente$, $m1 \mapsto A \in SurSite$ et $m1 \notin RERR$.

Ce scénario montre comment `DSR_5` modélise de façon abstraite la maintenance de routes : un message affecté par une rupture de lien est d'abord marqué dans `RERR` et mémorise le lien fautif dans `ErrLink`, puis il peut être réinjecté à la source pour un nouvel envoi.

3.6.7 Rôle de `DSR_5` dans le développement

Le raffinement `DSR_5` complète le modèle en introduisant un **mécanisme de maintenance de routes**, fondé sur le marquage des messages en erreur et sur la possibilité pour la source de les réinjecter dans le protocole.

Ce raffinement n'altère pas la logique de découverte de route déjà présente dans `DSR_3` et `DSR_4`, mais **enrichit la spécification** par l'ajout d'informations supplémentaires (`RERR` et `ErrLink`), rapprochant ainsi le modèle formel du comportement informel du protocole DSR, tout en maintenant une **charge de preuve maîtrisable**.

Dans notre modélisation, la structure de la table de routage `Cache` **n'a pas été modifiée** : *chaque destination est associée à une unique route*. Ce choix permet de conserver une architecture simple et cohérente avec les raffinements précédents.

Dans ce contexte, l'information portée par `ErrLink` n'est pas exploitée pour effectuer un choix dynamique entre plusieurs routes alternatives.

Elle constitue néanmoins une **extension pertinente du modèle** : si plusieurs routes par destination étaient autorisées dans `Cache`, `ErrLink` permettrait d'identifier un saut défaillant et d'éliminer de manière plus fine les routes contenant ce lien.

4 Améliorations et évolutions

4.1 Améliorations du modèle actuel

Nous avons choisi de ne modéliser qu'une seule route par destination dans les caches locaux, or il serait judicieux d'avoir plusieurs routes pour pouvoir choisir une route sans avoir à refaire une requête de route à chaque erreur lors de la transmission du paquet. Pour se faire il aurait fallu autoriser les sites à recevoir plusieurs fois une même `RREQ` provenant de routes différentes et permettre ainsi de générer plusieurs `RREP`. De plus, nous n'avons pas implémenté le système de comptage de sauts dans les `RREQ` (car dans notre modèle, il était impossible d'avoir des cycles dans les routes) qui serait indispensable pour modéliser la possibilité d'avoir plusieurs routes pour une même destination dans le cache.

4.2 Evolutions possibles

Dans le modèle actuel, tous les liens possibles entre les sites sont identiques. Or, dans des applications réelles comme les réseaux entre téléphones portables, ordinateurs portables et ordinateurs de bureau, etc. il y a une notion de portée et de distance.

Pour rendre le modèle plus réaliste, il serait intéressant d'introduire une notion de *range* pour chaque site, par exemple avec trois niveaux : `low` (téléphone portable, bluetooth), `middle` (ordinateur portable, WiFi) et `high` (ordinateur de bureau, LoRa). Cette information pourrait ensuite être prise en compte dans la définition de l'ensemble `Reseau` : deux sites ne seraient connectés que si leurs portées respectives permettent effectivement une communication directe.

Cela permettrait de rapprocher davantage le modèle d'un déploiement concret et d'étudier l'impact des différences de portée sur la découverte et la maintenance des routes, tout en restant compatible avec la structure de raffinements déjà mise en place.

5 Bilan

5.1 Difficultés rencontrées

L'une des plus grandes difficultés a été de comprendre correctement les exigences et d'ensuite parvenir à modéliser un système prouvable correspondant aux exigences.

De plus, nous avons eu du mal à créer des modèles initiaux simples. La première instance de DSR_0 que nous avions implémentée été trop complexe pour être prouvée correctement et ne s'alignait pas avec le principe de raffinement progressif.

Une autre difficulté majeure concernait les Proof Obligations. Même lorsque le comportement envisagé semblait correct sur le papier, l'implémentation du modèle faisait apparaître de nombreuses PO non prouvées, affichées en rouge.

Certains choix de modélisation rendaient le système excessivement complexe, augmentant ainsi significativement le nombre et la difficulté des obligations de preuve.

Nous avons donc dû simplifier notre modélisation, ajuster notre stratégie de preuve, et répartir les fonctionnalités sur plusieurs machines (DSR_0, DSR_1, . . ., DSR_5), afin d'obtenir à chaque étape des modèles plus légers, prouvables et cohérents avec la démarche de raffinement.

5.2 Bilans personnels

Ce projet nous a donné l'occasion de tester réellement nos capacités de conception : il fallait imaginer une solution qui satisfasse à la fois les exigences fonctionnelles et les exigences de sûreté, ce qui demande un **haut niveau d'abstraction**.

Au départ, notre première approche de modélisation était **trop ambitieuse** : les modèles étaient très complexes, la plupart des Proof Obligations restaient non prouvées, et malgré plusieurs semaines de travail en TP, nous avons dû reconnaître que la direction choisie n'était pas la bonne. Nous avons donc pris la décision de **revoir notre stratégie de raffinement** et de simplifier l'architecture globale des machines DSR.

Avant de reconstruire, nous avons relu attentivement le cours et tenté de mieux maîtriser les outils mathématiques d'Event-B, afin de savoir comment les combiner de manière cohérente. Nous avons compris qu'une **compréhension approfondie du problème informel** est indispensable pour proposer un bon modèle formel.

La phase de modélisation, du modèle DSR_0 jusqu'à DSR_3, a été particulièrement difficile, car chaque version révélait de nouvelles failles dans notre raisonnement. À chaque raffinement, nous avons dû ajuster la solution, parfois pendant plusieurs jours, ce qui a demandé beaucoup de temps mais a aussi amélioré notre maîtrise de la plate-forme Rodin et notre compréhension d'Event-B.

Enfin, la validation et la vérification du modèle (preuve des obligations et animation avec ProB) ont été **très formatrices** : elles nous ont permis de mesurer précisément l'impact de chaque modification et de vérifier, étape par étape, que notre développement allait dans la bonne direction.

6 Conclusion

Conformément à la stratégie de développement retenue, nous avons réalisé cinq niveaux de raffinement, en incluant les modèles DSR_4 et DSR_5 comme amélioration de la solution initiale.

Il manque certaines obligations de preuve notamment celles garantissant la continuité des routes et l'absence de boucle dans celles-ci.

Dans le paysage actuel des protocoles de routage, chaque protocole vise à résoudre un type de problème particulier. Dans ce projet, l'intérêt du protocole DSR réside dans sa capacité à permettre à un réseau ad hoc de s'auto-organiser et de se configurer sans infrastructure ni gestion centralisée, ce qui en fait un bon candidat pour illustrer les apports de la modélisation formelle.

La complexité de ce type de solution se reflète naturellement dans sa modélisation, qui constituait l'objectif principal de notre travail. Depuis les premiers choix de conception jusqu'à la mise au point des modèles et à leur vérification, en passant par nos raffinements successifs, nous avons souvent été bloqués, mais chaque difficulté a donné lieu à de nouvelles idées lors des séances de TP et grâce aux échanges avec **M. Dupont Guillaume**.

Nous avons appris qu'il faut parfois changer de point de vue, accepter de revenir en arrière et garder une attitude ouverte et positive : un échec n'est pas une perte de temps, car il apporte de l'expérience pour la tentative suivante.

Au fil du projet, nous sommes passés d'une phase de confusion et de tâtonnements à un véritable plaisir de construire et d'affiner les modèles, en appréciant le processus de modélisation lui-même.

Toutes ces étapes nous ont permis de mener à bien un développement formel sur un système complexe et ont renforcé notre envie de continuer à nous intéresser à ce domaine, ainsi que de réutiliser ces méthodes dans notre future pratique professionnelle.